Building data course — Week 5 — Haystack RDF and external timeseries references

sparql-playground / lessons

Generated for print and offline reading

Week 5 — Haystack RDF and external timeseries references

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. The scripts are in the repo next to each lesson. Keep notes under the ignored `student-work/` folder.

Contents

- Day 21 — Why hard-coded names break: one AHU's `mad_c` is the other's `mad_c_pct`
- Day 22 — Haystack tags as RDF: find a rule's inputs by tags with SPARQL
- Day 23 — UUIDs and the long table: stable ids, `samples`, and pivoting long to wide
- Day 24 — External timeseries references: one model, history in SQLite or in a CSV
- Day 25 — Model-driven FC1 through the app's API (week 5 challenge)

Day 21 — Why hard-coded names break: one AHU's `mad_c` is the other's `mad_c_pct`

Goal: see a rule that works on one AHU fail on the next, then fix it with the smallest possible model: a map from *role* ("outside-air damper") to *column* ("mad_c" or "mad_c_pct").

Read

Weeks 1–4 typed column names into every script: `supply_fan_status`, `da_p_inwc`, `mad_c`. That was honest and it worked, because we

picked columns that happen to exist under the same name on both AHUs. Not every column does. AHU_1's outside-air damper command is `mad_c`. AHU_2's is `mad_c_pct`. Same physical thing, same unit (percent open), different name in the BAS export. Ask both AHUs for `mad_c` and one of them fails:

	day21_pandas.py	**day21_sql.py**
ask for `mad_c` on AHU_2	`pd.read_csv(..., usecols=[..., "mad_c"])` raises `ValueError`	`SELECT mad_c FROM ahu_2` raises `no such column`

You can patch this with `if equip == "AHU_2": col = "mad_c_pct"`. Do that for 30 columns and 40 AHUs and the `if`s *are* the data model, written in code and copied into every script. A change of building means editing every script.

The way out is to keep the facts **out of the rule**. The rule says "I need the outside-air damper and the fan status". Something else, a **model**, says which column holds each. The smallest model is already in the data folder: `data/BUILDING_50/AHU_*/column_map.json`, a dictionary from role to column:

```
"outside-air-damper": "mad_c"          // AHU_1
"outside-air-damper": "mad_c_pct"      // AHU_2
```

Open-FDD calls the left-hand side a **role** (`fan-status`, `duct-static-pressure`, …). Now the script loops over AHUs, looks up the column for each role, and reads *that*. One script, any number of AHUs, and the only thing that changes between buildings is the map.

How each half uses the map:

Step	`day21_pandas.py`	
1. hard-coded attempt	`pd.read_csv(path, usecols=["mad_c"])` in a `try`	SE ..
2. role → column	`points["outside-air-damper"]`, from `column_map.json`	th th
3. load by role	`read_csv(usecols=[...]).rename(columns={damper_column: "oa_damper_pct"})`	th pa SE
3-4. summary	`running["oa_damper_pct"].mean()` …, one row per AHU in a loop	AV WH su = Al

Renaming vs building SQL. pandas just renames the loaded columns to shared names, so everything after the load is identical for both AHUs. SQL has no “rename after the fact”: the query text itself must differ per AHU, so the script *builds* it from the map.

That brings a rule you should never forget: a **value** can be a bound parameter (`:eq0`), but an **identifier** (a table or column name) cannot. `SELECT :col FROM t` returns the string, not the column. So identifiers get pasted into the SQL text, and text pasted into SQL is how SQL injection happens. The defence is a **whitelist**: accept a name only if it is a real column of the real table (`PRAGMA table_info`), then quote it (`"`, doubled if it appears inside). The SQL script does the whitelist check (step 2) before it pastes a name. Today the map is a file you trust, but the habit costs nothing and matters the first time a name comes from a user or a web form.

Run

```
uv run python lessons/course.py run 21
```

That runs `day21_pandas.py`, then `day21_sql.py`. It reads only the CSVs and `var/lessons.sqlite`. Both print the failed hard-coded attempt, the damper column per AHU and the summary:

```
Hard-coded `mad_c` on each AHU:
equipment_id  worked
       AHU_1    True
       AHU_2   False

Damper column per AHU, from
column_map.json:
equipment_id damper_column
       AHU_1         mad_c
       AHU_2     mad_c_pct

Outside-air damper while the supply fan
runs (one script, both AHUs):
equipment_id damper_column  run_rows
run_hours  mean_damper_pct  open_over_50_h
closed_h
       AHU_1         mad_c     18893
1574.42            42.88          605.83
357.83
       AHU_2     mad_c_pct     17995
1499.58            51.90          696.42
347.75
```

(SQL prints `1/0` where pandas prints `True`/`False`. The error *message* differs between pandas and SQLite; only whether it worked is compared.) While the fan runs, AHU_1's damper averages 43% open and AHU_2's 52%. Both spend about 350 running hours fully closed.

Try it

1. Add the mixed-air temperature (role `mixed-air-temp`) to both scripts and report its mean while the fan runs. Which lines change? Which don't?
2. In the SQL script, change one role's column in the map to `"mad_c; DROP TABLE ahu_1"` (just in the Python dict). What happens, and why is that good?

Answers

1. One more lookup (`mat_column = points["mixed-air-temp"]`, which is `mixed_air_temp_f` on both AHUs), one more name in `usecols` and the `rename` (pandas) or in the whitelist check and the `SELECT` (SQL), and one more number in the summary. The loop over AHUs, and everything that says *which AHU*, doesn't change. Adding a role is data, not control flow.
2. The whitelist check raises `ValueError: unknown column in column_map.json` before any SQL runs, because that string is not a column of the table. Without the whitelist, the string would have been pasted into the query text.

Why it matters

A column map is a model: it separates *what the rule needs* from *where this building keeps it*. But it is one JSON file per AHU, it only knows columns in one CSV, and it can't say that the supply fan is *part of* AHU_1 or that a sensor reads in °F. Tomorrow you meet a richer model: Haystack tags written as RDF.

Day 22 — Haystack tags as RDF: find a rule's inputs by tags with SPARQL

Goal: read one point's Haystack tags in `model/building_50.ttl`, then find FC1's three inputs for every AHU by asking for tags, in SPARQL (rdflib, pandas side) and in a flat SQL table (SQL side), and see what the graph holds that the table doesn't.

Read

Project Haystack describes equipment and points with **tags**. A **marker** tag is just a name (`point`, `sensor`, `pressure`, `his`); a **valued** tag has a value (`unit = inH2O`, `kind = Number`). Two valued tags are **refs**, pointers to other entities: `equipRef` ("this point belongs to that equipment") and `siteRef`. Tags are how people in the industry describe buildings, and this repo writes them as RDF, in its own `bts:` vocabulary, so a graph database can query them. One point, `bldg:AHU_1_DA_P`, in Turtle:

```
bldg:AHU_1_DA_P a bts:Entity ;
    rdfs:label "AHU1 DA-P" ;
    ref:hasExternalReference [ ... ] ;
# where its history lives (Day 24)
    bts:hasHaystackMarker "air",
"discharge", "his", "point", "pressure",
"sensor" ;
    bts:hasHaystackTag [ bts:tagName
"equipRef" ; bts:tagValue bldg:AHU_1 ],
                        [ bts:tagName
"unit" ;     bts:tagValue "inH2O" ],
                        [ bts:tagName
"ofddRole" ; bts:tagValue "duct-static-
pressure" ], ... .
```

Markers are one predicate with many values. Valued tags are small nodes with a `tagName` and a `tagValue`. `ofddRole` is this repo's tag for the Open-FDD role: the same role names as Day 21's map, now stored *in the model*, on the point. `equipRef`'s value is a node (`bldg:AHU_1`), not a string, so the graph can follow it.

Finding FC1's inputs. FC1 needs the fan status, duct static pressure and its setpoint, for each AHU. In SPARQL that is a pattern of tags with a `FILTER` on the role (the scripts ask for AHU_1 by putting `bldg:AHU_1` in place of `?equip`):

```
SELECT ?equip ?role ?point ?timeseries_id
WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseries_id
.
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ;
bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-
static-pressure", "duct-static-pressure-
sp"))
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ;
bts:tagValue ?equip .
}
```

Pattern order matters. rdflib evaluates triple patterns roughly top to bottom. The first pattern starts from the 118 points that have a timeseries reference, so every join after it is small. Start with "every node that has an `ofddRole` tag" and rdflib builds a large intermediate result first; against the API that hit its 10 s limit. A database planner reorders joins for you; a simple engine does not, so put the most selective pattern first.

The flat shadow. The model was built from `model/points/*.csv`, one row per point (point id, equipment, role, source column, UUID, …). The SQL script loads AHU_1's rows into an in-memory table `point` and asks the same

question as a plain `WHERE openfdd_role IN (...)`. The same three points come back. The SQL version is *simpler*. So why have a graph?

	flat table `point`	**RDF graph**
a point's role	a column, `openfdd_role`	a tag: `ofddRole`
which equipment	a column, derived here from the *file name*	a tag, `equipRef`, pointing at an entity
markers (`sensor`, `pressure`, …)	none: the CSV never had them	a first-class predicate
hierarchy (point → equipment → site)	one `owner_id` column, one level	refs you can follow to any depth
where the history lives	one column `timeseries_id`, one store	any number of references, each with its own store (Day 24)
a new kind of fact	`ALTER TABLE`	add a triple

The table is a **flat shadow**: it answers today's question fast, but it only contains what was flattened into it. The graph keeps the structure.

Step	`day22_pandas.py` (rdflib)
1. load	`Graph().parse("model/building_50.ttl"` (4,147 triples)
2. one point	its markers and tags
3. FC1's inputs	SPARQL `FILTER(?role IN ...)`
4. role count	`COUNT(?role)`

Run

```
uv run python lessons/course.py run 22
```

That runs `day22_pandas.py`, then `day22_sql.py`. No server is involved: rdflib reads the Turtle file straight into memory. The pandas script first prints one point's markers and tags; the SQL script prints the same point as a row (it has columns, but no markers). Both end with the same three rows:

```
FC1's inputs on AHU_1, found by role and
equipRef:
equipment_id                    role
point_id
timeseries_id
       AHU_1    duct-static-pressure
AHU_1_DA_P 52422584-9894-50bd-9696-
2fbb8b062518
       AHU_1 duct-static-pressure-sp
AHU_1_DA_P_SP 761c2b61-4982-5157-bc66-
2010cf6e2a47
       AHU_1              fan-status
AHU_1_SF_Status 9ebf80dd-7821-510b-b2c0-
cd0151e093c9

AHU_1 points with a fault-rule role: 13
```

Nothing in either script names a column. The two are compared on `(equipment, role, point id)` plus the UUID, and on the number of role-tagged points (13). Put `bldg:AHU_2` in the SPARQL (and AHU_2's CSV in the SQL) for the other AHU: 13 again.

Try it

1. Find the AHUs' **discharge air temperature** and its setpoint (`discharge-air-temp`, `discharge-air-temp-sp`) in both scripts. How many lines change?
2. In the pandas script, query by *marker* instead of role: which points have the

markers `pressure` **and** `sp`? Can the SQL table answer that?

Answers

1. Only the role strings in the filter (`FILTER(?role IN (...))` and `WHERE openfdd_role IN (...)`).
2. Two marker values on the same point: `?p bts:hasHaystackMarker "pressure", "sp" .` (the comma repeats the same subject and predicate). The flat table cannot: the CSVs have no markers. You could add a `markers` column, but then "has marker X" is a string search, where the graph has it as data.

Why it matters

Rules that ask for roles survive a change of building, as long as the building's model has the tags. That is why Haystack tagging is worth the effort in industry. Note what is still missing: the answer is a UUID, an opaque string you can't read. Tomorrow we look at where it comes from and at the table that stores history under it.

Day 23 — UUIDs and the long table: stable ids, `samples`, and pivoting long to wide

Goal: know why the history is stored under opaque, stable ids, see where Building 50's UUIDs come from, and pivot AHU_1's three FC1 inputs from the long `samples` table back into the CSV's columns.

Read

Why an opaque id. In the CSV a point's identity is its column name, `da_p_inwc`. That is fragile: the column is renamed when a controller is replaced, two buildings use the same name for different things (Day 21), and merging two exports collides names. A model-driven system wants an id with no meaning that never changes, so *everything else* (names, labels, units, tags) can change around it. Day 21's `mad_c` / `mad_c_pct` mismatch can't happen to an id nobody reads.

Where these ids come from. Each `timeseries_id` is a **UUIDv5**, defined in RFC 9562 (the 2024 update of RFC 4122): the SHA-1 hash of a namespace UUID plus a name, formatted as a UUID. The same name

always gives the same id, on any machine, with no registry to ask, so rebuilds and merges agree. This repo's rule:

```
uuid.uuid5(uuid.NAMESPACE_URL,
          f"urn:openfdd:{building}:
          {equipment}:{source_column}")
# urn:openfdd:BUILDING_50:AHU_1:da_p_inwc  ->
          52422584-9894-50bd-9696-2fbb8b062518
```

Both scripts recompute the id and compare it with the one in the point list. There is one trap: the id is derived from the *name once*, then **frozen** in `model/points/*.csv`. If you re-derived it at query time and someone renamed the column, the id would change and every stored series would be orphaned. The hash is a way to *mint* ids reproducibly, not a way to *find* them later. (A v4 UUID, random, would also work. You'd just have to keep the table that says which one is which, which is exactly what the frozen CSV is.)

The long table. The API's database, `var/timeseries.sqlite`, has no column names at all:

```
CREATE TABLE samples (timeseries_id TEXT,
          ts INTEGER, value REAL,
                       PRIMARY KEY
          (timeseries_id, ts)) WITHOUT
          ROWID
```

That is **long** (tall) form: one row per reading, 4.2 million rows for 118 points. The CSV is **wide**: one row per timestamp, one column per point. Long form has real advantages for storage. A new point is new rows, not a new column (`ALTER TABLE`); points that report at different times or with gaps don't need NULL padding; and the primary key `(timeseries_id, ts)` with `WITHOUT ROWID` stores each series contiguously and in time order, so "all of one series between two times" is one B-tree range scan. Wide form is what rules want: several signals side by side at the same timestamp.

Going from long to wide is a **pivot**: one output row per timestamp, one output column per series.

Step	day23_pandas.py
1. role → UUID	`pd.read_csv("model/points/BUILDIN(`
2. recompute the UUID	`uuid.uuid5(...)`
3. read the long rows	`SELECT ts, value FROM samples WHEI = ?`, once per point
4. pivot	`long.pivot(index="ts", columns="r values="value")`

Step	day23_pandas.py
5. compare to the CSV	merge on epoch seconds, == per column

`MAX(CASE ...)` is conditional aggregation again: within one timestamp each series has exactly one row, so `MAX` just picks it and the other rows' NULLs are ignored. pandas' `pivot` is stricter and raises if a `(ts, series)` pair repeats, which is a free check that your data has one value per timestamp. Either way it is O(n): one pass over the long rows, then a group per timestamp.

The wide result has to equal the CSV's own columns, because the database was filled from that CSV. The scripts check it value by value, and show the number of equal rows and the largest absolute difference.

Run

This day needs `var/timeseries.sqlite`: `uv run bldgts model build && uv run bldgts bootstrap` once. (The test suite skips the day if the file is missing.) The scripts open it read-only.

```
uv run python lessons/course.py run 23
```

That runs `day23_pandas.py`, then `day23_sql.py`. Both print (the wide table's columns come out in a different order, and its first rows are left out here):

```
The UUID of each FC1 input, and the same
UUID minted again from its name:
                   role      source_column
timeseries_id
recomputed_uuid  matches
   duct-static-pressure          da_p_inwc
52422584-9894-50bd-9696-2fbb8b062518
52422584-9894-50bd-9696-2fbb8b062518
True
duct-static-pressure-sp da_p_setpoint_inwc
761c2b61-4982-5157-bc66-2010cf6e2a47
761c2b61-4982-5157-bc66-2010cf6e2a47
True
             fan-status  supply_fan_status
9ebf80dd-7821-510b-b2c0-cd0151e093c9
9ebf80dd-7821-510b-b2c0-cd0151e093c9
True

Long table: 106608 rows for 3 series.
                   role  rows   first_ts
last_ts
   duct-static-pressure 35536 1773621600
1784282100
duct-static-pressure-sp 35536 1773621600
1784282100
             fan-status 35536 1773621600
1784282100

Compared with the CSV's own columns:
                   role      source_column
rows_compared  rows_equal  max_abs_diff
   duct-static-pressure          da_p_inwc
35536       35536           0.0
duct-static-pressure-sp da_p_setpoint_inwc
35536       35536           0.0
             fan-status  supply_fan_status
35536       35536           0.0
```

3 series × 35,536 rows = 106,608 long rows become 35,536 wide rows.

Try it

1. Change one letter of the name (`da_p_inwcX`) and mint the id again. How different is the result? What does that tell you about hand-editing ids?
2. Add the outside-air temperature to the pivot (role `outside-air-temp`) in both scripts. What does the long table cost per extra point, compared with adding a CSV column?

Answers

1. Completely different: a hash has no "nearby" ids. So an id can't be fixed by hand, and a rename means a new series unless you keep the old id. That is why the id is frozen in the point list.
2. One more role in the point filter and one more `MAX(CASE ...)` line (pandas' `pivot` adds the column by itself), and no schema change at all. The long table just has more rows. That is the storage argument for long form: points come and go without altering the table.

Why it matters

Now the model's answer ("the fan status is `9ebf80dd-...`") leads to a series, and you can rebuild the wide table a rule wants from the long one. The next question: the model says *an id*, but what says *where to look it up*? Tomorrow's answer is the external timeseries reference.

Day 24 — External timeseries references: one model, history in SQLite or in a CSV

Goal: see how the model says *where* a point's history lives, add a second kind of store (a CSV file) to an in-memory copy of the graph, write a resolver that reads a series from whichever store a reference names, and check that both stores give the same series.

Read

What plain Haystack leaves out. Haystack marks a point as having history with the marker `his` and reads it by point id (the `hisRead` operation: "give me the history of point X"). But it has no standard tag for *where* that history is stored. Which database? Which table, key, file? That is left to the server you ask.

What the ref schema adds. Brick's **ref schema** (the page "Timeseries storage" in the Brick docs) fills exactly that gap, and it doesn't care what the store is. A point has a `ref:hasExternalReference` to a

`ref:TimeseriesReference`. The reference has exactly one `ref:hasTimeseriesId` (a string: the key of the series in the store) and a `ref:storedAt` pointing at the store, which Brick types as a `ref:Database`. Brick doesn't say what a database node must contain. This repo combines the two vocabularies, Haystack tags for *what the point is* and reference nodes for *where its data is*:

```
bldg:AHU_1_DA_P  bts:hasHaystackMarker
"his", "point", ... ;
    ref:hasExternalReference [ a
ref:TimeseriesReference ;
        ref:hasTimeseriesId "52422584-
9894-50bd-9696-2fbb8b062518" ;
        ref:storedAt bldg:timeseries_db ]
.

bldg:timeseries_db a ref:Database ;
    bts:backend "sqlite" ;
bts:connectionEnv "BLDGTS_DB_PATH" ;
    bts:schemaDescription
"samples(timeseries_id TEXT, ts INTEGER
epoch-s UTC, value REAL)" .
```

So the model has, for every point, "ask `timeseries_db` (SQLite; path in `$BLDGTS_DB_PATH`) for id `52422584-...`". Because the reference is a node, a point can have **several**: the same sensor's history in the live database and in an archive, or in a file someone sent you.

A second store. Today we add one, in memory, without touching `model/building_50.ttl`. Each AHU's CSV export becomes a store, and each point gets a second reference to it. A CSV has no UUIDs: a series is identified by its column, so that CSV reference's `hasTimeseriesId` is the **column name** (`da_p_inwc`). Where does the script learn which column goes with which role? From Day 21's `column_map.json`. The model's roles plus the map give the column.

```
bldg:csv_AHU_1 a bts:CsvFile ;  bts:path
"data/BUILDING_50/AHU_1/history_wide.csv"
;

bts:timeColumn "timestamp_utc" .
bldg:AHU_1_DA_P ref:hasExternalReference [
a ref:TimeseriesReference ;
        ref:hasTimeseriesId "da_p_inwc" ;
ref:storedAt bldg:csv_AHU_1 ] .
```

The store gets its own class, `bts:CsvFile`, and not `ref:Database`. Brick's `ref:Database` represents a storage *system*; calling a file a "database" would stretch that class's meaning, and anything that dispatches on the class ("a database: connect to it") would be wrong about a file. A class of our own keeps the dispatch honest.

The resolver is the one function that knows about stores. Given a reference it looks at the store's class and reads the series from the right place:

Store class	day24_pandas.py	d
ref:Database (sqlite)	read_sql_query("SELECT ts, value FROM samples WHERE timeseries_id = ?")	the sam ATTACHe
bts:CsvFile	read_csv(path, usecols=[time, column]), time text → epoch seconds	load the into a TE unixepc as ts
anything else	raise ValueError	raise V

Notice what stays the same: the rule asks the model for "the fan status of AHU_1" and gets back references; the resolver picks the store. Nothing in the rule knows whether the data came from a database or a file. Swapping storage means changing the model (and, for a brand-new store *type*, adding a branch to the resolver), not the rules.

One gotcha. Once a point has two references, a query that joins through `ref:hasExternalReference` returns that point *twice* (once per reference). The scripts use `SELECT DISTINCT ?point` to find the points and

then ask for their references separately. If a result suddenly has twice the rows after you add a reference, this is why.

The scripts compare the two stores for each of the 6 FC1 input series (3 per AHU): same number of rows, same values?

Run

Needs `var/timeseries.sqlite` (`uv run bldgts model build && uv run bldgts bootstrap`).

```
uv run python lessons/course.py run 24
```

That runs `day24_pandas.py`, then `day24_sql.py`. Both build the same graph with rdflib (SQL can't read RDF), then differ in how they read the series. Both print:

```
       point_id         store store_class
timeseries_id
     AHU_1_DA_P     csv_AHU_1     CsvFile
da_p_inwc
     AHU_1_DA_P timeseries_db    Database
52422584-9894-50bd-9696-2fbb8b062518
     ...
       point_id  rows_sqlite  rows_csv
rows_equal  max_abs_diff
     AHU_1_DA_P        35536     35536
35536           0.0
     ...
```

All six series have 35,536 rows in each store and every value is identical.

Try it

1. Add a third reference to one point: another `ref:Database`-typed node, with `bts:backend "postgres"`. What does the resolver do when it meets it? Is that the right behaviour?
2. Make the resolver pick the store: add a parameter `prefer` ("Database" or "CsvFile") that chooses among a point's references. What does a rule need to know to use it?

Answers

1. `read_series` raises `ValueError: no reader for database backend 'postgres'`. That is the right behaviour: failing loudly beats returning an empty series. To support it you add one branch (and a driver).
2. Nothing: `prefer` is part of the resolver call, not of the rule. Today's `refs` table has `store_class`; filter it before `read_series`. Day 25's challenge does exactly this.

Why it matters

This is why model-driven FDD survives a change of storage. When history moves from CSV to SQLite, or SQLite to a real time-series database, you change the *references*. The rules, written against roles and tags, don't move. Tomorrow's rule runs through the app's API, which does the resolving for us.

Day 25 — Model-driven FC1 through the app's API (week 5 challenge)

Goal: run Day 14's FC1 rule with no column names and no ids written in the code: SPARQL finds the inputs, the app's API (or SQL on its database) supplies the series. Then run the same rule against the CSV storage from Day 24.

Read

Day 14 hard-coded `supply_fan_status`, `da_p_inwc` and `da_p_setpoint_inwc`. This week replaced each hard-coded fact with a lookup: roles (Day 21), tags in RDF (Day 22), stable ids (Day 23), references to stores (Day 24). Today they all work together.

FastAPI's `TestClient` runs this repo's app (`bldgts serve`) inside the Python process: requests go straight into the app, no server and no network. The app reads the model `model/building_50.ttl` and the database `var/timeseries.sqlite`; the pandas script starts it read-only (`allow_mutations=False`). We use two of its routes:

- `POST /api/sparql` runs SPARQL against the model and returns the standard SPARQL JSON results. The query is Day 22's: roles and `equipRef`, starting from points that have a timeseries reference (pattern order, as before).
- `GET /api/points/{id}/timeseries`: the API looks up the point's reference, resolves it (the Day 24 job, done server-side) and returns its samples, `ts, value`, plus the `timeseries_id` it served.

Day 14's rule is unchanged. FC1 is the same four steps (gate and raw fault, streak id, row in streak, confirm) with the same tolerance (25 Pa) and confirmation (2 rows), now on both AHUs. The only new work is **getting the data in the shape the rule wants**, long → wide (Day 23) and inches of water → pascals (Day 6):

Step	`day25_pandas.py`	`day25_sql.py`
1. find the inputs	`api.post("/api/sparql", json={"query": fc1_query})`	the same query, with rdflib on the model file
2. get the series	`api.get("/api/points/" + point_id + "/timeseries")` ×6, one `GET` each	none: SQLite reads `samples` directly (read-only)

Step	day25_pandas.py	day25_sql.py
use the ids	the API is told the point; it returns the UUID it served (the script prints it equals the model's)	UUIDs go into a TEMP table fc1_input, joined to samples
3. long → wide	long.pivot(index=["equipment_id", "ts"], columns="role", values="value")	MAX(CASE WHEN i.role = 'fan-status' THEN s.value END) ...
4-5. SI + FC1	pandas arithmetic, shift() / cumsum() / cumcount()	LAG, SUM() OVER, ROW_NUMBER() OVER, all PARTITION BY

Both routes have the same cost shape: O(n) rows moved and one sort for the windows. Through HTTP each series is also serialized to JSON and back, so the API route is slower per row than reading SQLite directly. You pay for the indirection to get a stable interface: the client never sees the database, the table or the UUIDs it didn't ask for. The SQL script shows the other side: when you do own the database, the model's answer ("these UUIDs") goes into a table the query joins, and no id is written in the SQL.

Where do the role names (“fan-status”, …) still appear? In the SPARQL filter and the pivot (the SQL’s `CASE WHEN i.role = ...` lines, pandas’ column names). They are the rule’s **input contract**: that is all the rule knows about the building. Everything else, which point, which column, which store, which id, comes from the model.

Run

Needs `var/timeseries.sqlite` and the model (`uv run bldgts model build && uv run bldgts bootstrap`).

```
uv run python lessons/course.py run 25
```

That runs `day25_pandas.py`, then `day25_sql.py`. The pandas script first prints one line per `GET` (`GET AHU_1_DA_P -> 35536 rows, same id as the model: True`, …). It may also print a `DeprecationWarning` from Starlette, the web library under FastAPI; it is harmless. Both scripts end with:

```
FC1's inputs, found by the model:
equipment_id                    role
point_id
timeseries_id
       AHU_1    duct-static-pressure
AHU_1_DA_P 52422584-9894-50bd-9696-
2fbb8b062518
       AHU_1 duct-static-pressure-sp
AHU_1_DA_P_SP 761c2b61-4982-5157-bc66-
2010cf6e2a47
       AHU_1              fan-status
AHU_1_SF_Status 9ebf80dd-7821-510b-b2c0-
cd0151e093c9
       AHU_2    duct-static-pressure
AHU_2_DA_P f8a0d594-2fff-567a-9853-
71ba66233eba
       AHU_2 duct-static-pressure-sp
AHU_2_DA_P_SP 0c281cac-4d5e-5892-9456-
bb9d1038fae8
       AHU_2              fan-status
AHU_2_SF_Status 4bac9a08-337e-5f6b-a577-
fa044776ab05

Rows after the pivot: 71072

FC1 per AHU (hours, episodes). Day 14 got
the same from hard-coded columns:
equipment_id  raw_fault_h
confirmed_fault_h  episodes
       AHU_1       120.00
114.25        53
       AHU_2         3.42
1.50         5
```

These are Day 14's numbers (AHU_1 114.25 h in 53 episodes; AHU_2 1.50 h when Day 14 is pointed at AHU_2), from scripts that name no column. The per-row fault flags are compared too, so the two scripts agree on every one of the 71,072 rows.

Try it

1. Add the **duct static too high** rule (Day 14, Try it 2) to the pandas script. Which part of the pipeline changes, and does it need any new input from the model?
2. Run the script for AHU_2's `discharge-air-temp` and `discharge-air-temp-sp` instead of the static pressure roles. What do you change? (Mind the units: they are °F in the database.)

Answers

1. Only the comparison (`>` and `+ TOLERANCE_PA`). It uses the same three roles, so no new input.
2. The role strings in the SPARQL `FILTER` and the role column names used after the pivot; and the conversion: temperatures use `(F - 32) * 5 / 9`, not `* 249.08891`. The tolerance would be a temperature *difference*, which has no 32 in it (Day 6).

Week 5 challenge: the same rule, a different store

Day 24 gave every FC1 input point a second reference, to the AHU's CSV. Run FC1 **again, with the series read from the CSV storage**, using the model to find the points and Day 24's resolver to read them. You should get AHU_1 114.25 h in 53 episodes and AHU_2 1.50 h again. Try it before opening the answer.

Answer

Start from a copy of `day24_pandas.py`: it already adds the CSV references to the graph, and its step 4 (the resolver) has read every FC1 input from both stores into the `series` dict, keyed by `(point_id, store_class)`. Append this. It changes exactly one thing, which store's series the rule uses (`STORE`); the rest is Day 25's:

```
PA_PER_INH2O = 249.08891
TOLERANCE_PA = 25
CONFIRM_ROWS = 2
STORE = "CsvFile"  # <- the only change
        from Day 25: which store's
        series to use

# Each FC1 input point, with its AHU and
        role (DISTINCT: one row per
        point)
roles_query = (
    PREFIXES
    + """
SELECT DISTINCT ?point ?equip ?role WHERE
        {
  ?point bts:hasHaystackTag ?role_tag .
  ?role_tag bts:tagName "ofddRole" ;
        bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-
        static-pressure", "duct-static-
        pressure-sp"))
  ?point bts:hasHaystackTag ?equip_tag .
  ?equip_tag bts:tagName "equipRef" ;
        bts:tagValue ?equip .
}
"""
)
pieces = []
for row in g.query(roles_query):
    point_id = str(row.point).split("#")
        [-1]
    one_series = series[(point_id,
        STORE)].copy()  # step 4's
        resolver already read it
    one_series["equipment_id"] =
        str(row.equip).split("#")[-1]
    one_series["role"] = str(row.role)
    pieces.append(one_series)

long = pd.concat(pieces)
```

```
wide = long.pivot(index=["equipment_id",
         "ts"], columns="role",
         values="value").reset_index()
# ... then Day 25's steps 4-5, unchanged:
         pascals, raw fault, streaks,
         hours and episodes.
```

The CSV series are in the same units and on the same timestamps as the database's, so the rule gives the same answer: AHU_1 114.25 h, 53 episodes; AHU_2 1.50 h. Without the `DISTINCT`, a query that also matched the references would return each point once per reference, and the pivot would raise "Index contains duplicate entries". For the SQL route, load the CSV into a `TEMP` table as `day24_sql.py` does and join it instead of `samples`.

The lesson: the rule, the SPARQL and the pivot did not change. Only the store the resolver was told to prefer did. That is what the model buys you.

Next: **Unit 4** goes deeper into graphs and SPARQL itself, then the BACnet scans that feed models like this one.

Reading — Week 5

The lessons first, then these for exact behavior.

- Brick: timeseries storage (ref schema)
- Project Haystack: defs
- Project Haystack: RDF
- RFC 9562: UUIDs (version 5 = name-based)
- rdflib documentation
- FastAPI testing (TestClient)